

CHAKRAVYUH — Build Plan

Team Tech Coders · Team ID 7-300 · Problem Statement FC-02 · Fintech & Cyber

What we are building

We are building **Chakravyuh** — a system that reads a bank's transactions, works out who the people behind the accounts really are, finds groups working together to launder money, and writes up the case for a human officer to approve.

Today a bank's software is a security guard who checks one person at a time at the gate. Everybody looks fine individually, so everybody walks through. Our system is a guard who watches the whole crowd and notices that six people keep passing the same bag between them — and that three of those six are the same man under different names.

What it does:

- Works out who is who.** The same person sits in the bank's records three times over, written differently each time. We merge them into real people before doing anything else.
- Finds the group, not the person.** It looks at how money moves between people and spots shapes only criminals make — circles, sprays, money that never rests, amounts hugging the reporting limit.
- Writes the case, not an alert.** The officer gets a finished report: what happened, which accounts, how much, in what order, and why it is suspicious.
- Watches known bad actors on a lower threshold.** A criminal we already know about may move money in a shape that looks innocent. Known people get extra scrutiny.
- Lets two banks find a shared gang without sharing customer data.** Only scrambled account codes and the shape of the flow — never names, never balances.

The user is a compliance officer at a bank. Today they get hundreds of alerts and almost all are wrong. We give them three real cases instead.

Why we are building it

The problem	The number
Money laundered worldwide each year	2-5% of global GDP (\$800bn-\$2tn)
How much authorities actually catch	under 1%
What banks spend on compliance each year	\$206 billion
Alerts that are false alarms	85-95%
Alerts that become a real report	1-5%
Officer time spent on alerts leading nowhere	up to 90%

Criminals never move dirty money in one big suspicious transfer. They break it into many small, boring, legal-looking transfers. Bank software checks one transfer at a time, so each passes. The crime is only visible when you look at all of them together — and nobody's software does that.

TD Bank failed to check 92% of their transactions, three gangs pushed \$670 million through them, and their rules had barely changed since 2014. They were not short of alerts. They could not connect them.

In India this is live: banks have frozen around 8.5 lakh mule accounts, and the blunt approach also freezes students and small traders who did nothing wrong.

Roughly \$1.1 trillion of laundered money is drug money and \$528 billion is human trafficking money. Laundering is what makes those crimes worth committing.

The system, stage by stage

Eight stages. Two use AI; the rest are plain logic, on purpose.

1. Bank data	transactions + records from 3 systems
↓	
2. Entity resolution	scattered records → real people
↓	↳ watchlist matches noted here
3. The map	people = dots, money = arrows
↓	
4. Detectors ×4	circle, spray, speed, threshold
↓	
5. Case builder	seed → trace → expand → prune
↓	↳ two lanes: discovery + watchlist
6. Investigator AI	writes the case, names the method
↓	

7. Verifier AI	attacks it, checks every claim
↓	
8. Human officer	approves → STR filed

Stage 1 — The data

What: A made-up but realistic set of Indian bank transactions — about 12,300 over 30 days across two banks — plus customer identity records from three separate systems, with four laundering rings hidden inside.

Why: No bank will give us real customer data. Planting the rings ourselves also means we know the right answer, so we can prove how accurate we are. Real AML systems train on filed reports, which are analyst opinions, not convictions. Our ground truth is cleaner.

How: generate_data.py. Fixed random seed, so every run is identical and the demo is reproducible.

Stage 2 — Entity resolution

What: Work out who is actually who. The same person sits in the records three times — "Karthik Pillai" in retail, "KARTHIK PILLAI" in corporate, "Pillai Karthik" on a watchlist — sometimes with a phone number, sometimes a PAN, sometimes neither. This merges them into one real person.

Why: Without it the map is wrong. Six accounts that look like six strangers may be three people, and the circle is invisible because its dots are fragmented. In our data, **47 ring accounts are controlled by only 19 real people**. The ring is only a ring after resolution.

This stage also produces the **watchlist lane**: some resolved people match a known bad actor. Noted here, traced later once the map exists.

How: Match on shared phone, PAN and address, plus fuzzy matching on name spellings. Plain rules, no AI — every merge must be explainable.

Stage 3 — The map

What: Turn the transactions into a map. Every resolved person is a dot. Every transfer is an arrow carrying amount and time.

Why: This is the whole idea. A list hides the crime; a map shows it. You cannot see a circle in a spreadsheet.

How: A standard Python graph library. Solved problem, not something we invent.

Stage 4 — The detectors

Four small programs, each hunting one shape.

Detector	What it hunts	Why it matters
Circle	Money that leaves and comes back	No honest business sends money in a loop
Spray	One sending to many, or many feeding one	The classic mule pattern
Speed	Money leaving within minutes of arriving	Real money rests for days; dirty money runs
Threshold	Transfers sitting just under the reporting limit	Nobody accidentally sends ₹9.8 lakh eleven times

Why four: One signal alone is weak. A circle might be innocent. A circle *plus* speed *plus* threshold-hugging is not.

How: Plain maths and graph rules, no AI. Deliberate — results must be exact, repeatable and explainable to a regulator.

Stage 5 — The case builder

What: Four steps. Take each flagged person as a **seed**. **Trace** the money in and out. **Expand** to whoever they touch. Then **prune** — throw away every innocent account, so the case contains only the guilty network plus one ring of context.

Why: Without this you get four separate alerts about one crime. This turns noise into a case. The one ring of context stays because an analyst will always ask where the money entered from and where it went.

Two lanes arrive here and run side by side:

Lane	What it holds	Where it starts
Discovery	Rings the detectors found alone, nobody known in them	A pattern in the data
Watchlist	Known bad actors and their money links, lower threshold	A name we already know

The watchlist lane exists because a criminal we already know about may move money in a shape that looks completely innocent — one clean transfer to a new account, nothing for a detector to catch. So known people are judged on weaker signals too: a counterparty never seen before, money into a freshly opened account, any break from normal pattern. The industry name is **risk-based monitoring**, and enhanced due diligence on high-risk customers is what FATF and the RBI's KYC directions require.

If a discovery case and a watchlist case share accounts, they merge into one case.

Stage 6 — The investigator (AI)

What: The first place we use AI. It receives one confirmed network and writes the case: what happened, in what order, how much moved, which laundering method it matches, and what the officer should do.

Why: This is the part that takes a human four hours, and the only part needing judgement and language.

How: We send the AI the network and its evidence, instructed to describe only what is in front of it. Translator, not detective. Detection

already happened in Stage 4.

Stage 7 — The verifier (AI)

What: A second AI whose only job is to attack the first one's report. Re-checks every number against raw transactions, looks for an innocent explanation, returns pass / revise / reject with a confidence level.

Why: An AI inventing a detail inside a legal filing would be a disaster. This is the safety net, and nobody else will have built it.

How: Same AI, different instructions — a sceptic rather than a writer.

Stage 8 — The human

The officer sees the case, the map and the confidence level, and clicks approve or reject. **The system never files anything by itself.**

The demo story

Everything we build serves these ninety seconds.

1. **The mess.** 12,287 transactions, 347 accounts. Nothing looks wrong. "A bank's normal month."
2. **The fragments.** The same man written three ways across three systems. "The bank does not know these are one person."
3. **Resolve.** Accounts collapse into real people. **The circle appears.** It was always there — the bank just could not see it was the same people.
4. **Press Run.** Detectors light up: Circle 3, Spray 1, Speed 7, Threshold 12. Two seconds.
5. **The prune.** The map greys out until only six people and the loop remain.
6. **The reveal.** ₹2.4 crore round a closed loop in four hours, crossing both banks.
7. **The case writes itself.** Plain English: accounts, amounts, order, method.
8. **The verifier speaks.** "Checked 9 claims against raw data. All confirmed. Confidence high."
9. **The human decides.** Approve → STR generated.

Strongest beat is step 3. Closing line: *"Three hundred alerts became three cases. And the officer did not have to write a word."*

Cut list

Cut first: the graph picture. Show the network as a clean table instead.

Cut second: the two-lane view. One case list with a "known actor" tag. The watchlist logic still runs underneath.

Cut third: the cross-bank moment, then two of the four detectors.

Never cut: the fake data, entity resolution, the Circle detector, the written case, the human approve button. Entity resolution has a bypass switch — a failure there drops back to raw accounts rather than stopping everything.

Permanently out of scope: login, sign-up, user accounts, settings, admin panel, mobile app, live bank connections, crypto.

Decisions made

Decision	Settled on
Name	Chakravyuh
Stack	Python + Streamlit, built with Claude Code in this folder
Detection	Plain graph logic, never AI — reproducible and auditable
AI agents	Two only: investigator and verifier
Entry points	Two lanes, discovery and watchlist, merged when they overlap
Data	Synthetic with injected rings, so we have ground truth

Rules we do not break

1. **Save every time something works.** A bad hour should cost one hour, never the night.
2. **One change per request.** Ask, check, save, next.
3. **Three strikes and cut.** If a bug survives three fix attempts, remove the feature or fake its output.
4. **Never say "fix everything" or "refactor."**
5. **Deploy early, redeploy often.**
6. **Nothing new after feature freeze.**
7. **Record the demo video before sleeping.**

Answers to have ready

"What if the criminal isn't on any watchlist?" The detectors sweep the whole map regardless. The watchlist is a shortcut, not a requirement.

"Isn't this just an LLM wrapper?" The LLM does no detection. Graph analysis finds the structure and proves it; the LLM only turns a verified finding into language. The evidence exists before the model speaks.

"Lower thresholds on known people means harassing anyone wrongly listed." The extra scrutiny produces a case for a human, never an automatic freeze. Every case carries a confidence score and its evidence, so a weak one is dismissed in seconds. Every decision is logged, so repeated flagging with nothing found is visible and reviewable — more than a rule engine offers today.

"Your data is fake." Synthetic by design, with rings injected at known positions so we can measure accuracy. Real systems train on filed reports, which are analyst opinions rather than convictions. Our ground truth is cleaner than theirs.